Bangladesh University of Business and Technology (BUBT)

PROJECT PROPOSAL

On

Sudoku Master Pro

A Console-Based Sudoku Management System

[Submission Date: 18th, June, 2026]

Submitted By:

ID	Name	Intake/Section
20254103248	OALID KHAN	56/07
20254103245	MUNTAHA MOU JIM	56/07
20254103257	ANTAR CHANDRA KAR	56/07

Submitted To

Mastura Sadaf

Lecturer, Dept. of CSE

BUBT

1. Needs/Problems

The growing demand for digital puzzle games and educational brain-training applications highlights a significant gap: most Sudoku applications are mobile-only, require internet connectivity, or are built as bloated GUI applications unsuitable for academic or resource-constrained environments. The following core problems motivate this project:

- Lack of offline, lightweight Sudoku tools that run entirely in a terminal or console environment without any third-party runtime dependencies.
- Absence of a structured multi-user system that supports individual accounts, persistent saved games, and per-user statistical tracking within a console application.
- No readily available open-source C-language Sudoku system that demonstrates all core software-development concepts — file handling, recursion, backtracking, modular programming, and data structures — in a single cohesive project.
- Existing systems lack transparency in scoring: players cannot see exactly how wrong moves, hint usage, and elapsed time affect their ranking.
- Students learning C programming have no reference implementation showing how to build a complete, menu-driven, file-persistent application suitable for university Software Development Project (SDP) submissions.

2. Goals/Objectives

Sudoku Master Pro is a fully console-based Sudoku Management System written in pure C language, designed to run on any POSIX or Windows platform with GCC. The primary objectives are:

1. Implement a secure, file-backed multi-user authentication system (Sign Up / Sign In / Logout) with username and password validation.
2. Deliver three difficulty levels — Easy, Medium, and Hard — each with distinct puzzle complexity and a calibrated hint budget (5 / 3 / 1 hints respectively).
3. Provide a fully functional 9×9 Sudoku board with ASCII art display, protected original cells, real-time move validation using Sudoku rules, and an integrated backtracking solver.
4. Implement a transparent, formula-driven scoring system combining base score, wrong-move penalties, hint penalties, time penalties, and completion/no-hint bonuses.
5. Build a persistent save/load system supporting up to 5 save slots per session, storing complete board state, timer, score, difficulty, and hint count.

6. Maintain a Top-10 global high-score leaderboard and per-user statistics (games played, win rate, best score, best time, hints used).
7. Include an in-application tutorial explaining Sudoku rules, controls, and the scoring system to onboard new players.
8. Produce a well-commented, modular codebase (1300+ lines) that serves as a reference-quality university SDP submission.

3. Existing System Analysis

Several Sudoku applications already exist. Three representative systems are analyzed below:

3.1 Gnome Sudoku (Linux Desktop GUI)

Gnome Sudoku is an open-source graphical Sudoku application bundled with the GNOME desktop environment. It offers multiple difficulty levels, an undo system, and basic hint functionality. However, it requires a full graphical desktop environment, is platform-specific to Linux/GNOME, and cannot be run in a terminal session or on headless servers. It has no multi-user support and stores a single profile per OS user, making it unsuitable for demonstrating file-handling and structure-based persistence in C.

Shortcomings: GUI-only, no console mode, no multi-user authentication, no transparent scoring formula, no leaderboard, GNOME dependency.

3.2 Web-Based Sudoku.com

Sudoku.com is a popular browser-based Sudoku platform offering daily puzzles, four difficulty levels, and a timer. While feature-rich, it requires permanent internet connectivity and a modern web browser. It offers no offline mode, exposes no scoring formula to the user, and requires a server-side account. The JavaScript/React front-end and Node.js back-end architecture make it irrelevant as a C programming study reference. High-score data is server-side and inaccessible for local academic use.

Shortcomings: Requires internet, not written in C, closed-source, no offline file persistence, scoring is opaque to the end user.

3.3 ncurses-sudoku (Terminal, C)

ncurses-sudoku is a minimalist terminal Sudoku game written in C using the ncurses library. It demonstrates a console interface but requires the ncurses library to be installed, limiting portability. It has no user authentication, no save/load system, no leaderboard, no statistics tracking, and no tutorial. The single-file implementation does not demonstrate modular programming principles expected in a university SDP.

Shortcomings: ncurses dependency, single-user only, no persistence, no scoring, no leaderboard, poor modularity.

Sudoku Master Pro addresses all of the above shortcomings: it runs in any standard C console with no external libraries, supports multiple users with file-backed persistence, provides a transparent scoring formula, and demonstrates all SDP programming concepts in a modular, well-commented codebase.

4. Features / Scopes

The following features are confirmed deliverables within the project timeline:

- User Authentication: Sign Up, Sign In, Logout with file-backed storage (users.dat), username uniqueness validation, and minimum password length enforcement.
- Main Menu: Seven-option menu (New Game, Continue, High Scores, Tutorial, Statistics, Logout, Exit) with input validation.
- Difficulty Selection: Three levels (Easy / Medium / Hard) with 5, 3, and 1 hint allocations respectively and proportional puzzle complexity.
- 9×9 Sudoku Board: ASCII-art board with row/column indices, locked original cells, move entry by (row, col, number) coordinates, and dual-style cell markers.
- Backtracking Solver: Recursive solveSudoku() function used for solution generation, hint reveals, and optional auto-solve.
- Hint System: Reveals one random empty cell from the solution; deducts hint count and applies score penalty.
- Save / Load System: Up to 5 save slots per binary file (saves.dat), storing full board, solution, locked mask, score, timer, hints, and difficulty.
- Score System: Formula-driven — Base(1000) − wrong×10 − hints×50 − time/10 + completion(+500) + no-hint bonus(+200).
- Real-Time Timer: Elapsed time displayed on board; used in scoring and high-score records.
- Top-10 Leaderboard: Persisted in highscores.dat, sorted by descending score, displayed in tabular format.
- Profile Statistics: Per-user record in statistics.dat — games played, win rate, easy/medium/hard completions, best score, best time, total hints.
- Tutorial: In-app text tutorial covering rules, controls, scoring, and board legend.
- Game Completion Screen: Displays difficulty, time, hints used, final score, and NEW HIGH SCORE notification if applicable.

5. Development Model

The project adopts the Incremental Development Model for the following reasons:

- The system can be partitioned into independently testable modules: authentication, game engine, solver, file persistence, scoring, and UI — making incremental delivery natural.

- Each increment produces a working executable, allowing continuous testing and feedback before the next layer is added.
- The backtracking solver and file I/O modules carry the highest technical risk; building them early in a dedicated increment reduces overall project risk.
- Team members can work on parallel increments (e.g., one member on the game engine while another works on the statistics module), improving productivity.
- Unlike the Waterfall model, the Incremental approach accommodates small requirement changes (e.g., adding a new puzzle bank or score formula tweak) without restarting the entire development cycle.

The planned increments are: (1) Project setup and authentication module, (2) Board display and move validation, (3) Backtracking solver and hint system, (4) Save/load and file persistence, (5) Scoring, leaderboard and statistics, (6) Tutorial and UI polish, (7) Final integration testing and submission.

6. Feasibility Analysis & Possible Risks

Factor	Feasibility Assessment	Risks & Mitigation
Technical Feasibility	All features are implementable in ANSI C using standard libraries only (stdio.h, stdlib.h, string.h, time.h). No external dependencies are required. GCC is freely available on all target platforms.	Backtracking solver complexity; binary file portability across OS. Mitigated by thorough unit testing of solver and using fixed-size structs for file I/O.
Operational Feasibility	The application runs in any standard terminal window. No installation, GUI toolkit, or network connection is required. All team members are familiar with C programming from coursework.	Team members have varying C skill levels. Mitigated by assigning modules according to individual strengths and conducting weekly code reviews.
Economic Feasibility	Zero licensing cost. Development tools (GCC, VS Code, Git) are all free. No server or hosting costs. File storage is minimal (< 100 KB total).	Time overruns if the solver or file I/O modules prove more complex than estimated. Mitigated by 10% time buffer built into the schedule.

7. Requirements

Hardware Requirements

- Processor: Any x86 / x86-64 / ARM processor (no minimum specification)

- RAM: 4 MB minimum (application footprint is under 1 MB)
- Storage: 5 MB free disk space (executable + data files)
- Display: Any terminal or console window (minimum 80×25 characters)

Software Requirements

- Operating System: Windows 7+, Ubuntu 18.04+, macOS 10.14+, or any POSIX-compliant OS
- Compiler: GCC 7.0 or later (MinGW on Windows), ANSI C99 standard
- Build Tool: GNU Make (optional) or direct GCC invocation
- Text Editor / IDE: VS Code, Code::Blocks, Dev-C++, or any plain text editor
- Version Control: Git (GitHub / GitLab for repository hosting)

Programming & Technology Stack

- Language: C (C99 standard)
- Standard Libraries: stdio.h, stdlib.h, string.h, time.h, ctype.h
- Data Persistence: Binary flat-file I/O (fread / fwrite) with fixed-size structs
- Algorithm: Recursive backtracking for Sudoku solving and solution generation
- Paradigm: Procedural / modular programming

Personnel

- 3 student developers (roles defined in Section 11)
- 1 faculty supervisor (Mastura Sadaf, Lecturer, CSE, BUBT)

8. Activity and Time Schedule/Project Breakdown with time estimation

#	Stages/ Tasks	Efforts, man-hours
Stage 1	**Analysis and Design**	
1.1	Requirements analysis, mockup creation	18
1.2	Work plan creation	3
Stage 2	**Implementation**	
2.1	File system management	14

2.2	User authentication module	12
2.3	Board display and ASCII art UI	8
2.4	Move validation and game loop	10
2.5	Backtracking solver and puzzle bank	12
2.5	Hint system and score engine	10
2.6	Save / Load system with 5 slots	8
2.7	Leaderboard and statistics module	12
2.8	Tutorial section and completion screen	6
Stage 3	**Testing and other QA tasks**	
3.1	Testing and bug fixing	8
3.2	Unit testing of solver and file I/O modules	6
3.3	Cross-platform compilation test	4
Stage 4	**Deployment**	
4.1	Deployment on the hosting of client	3
	Total estimated efforts: 134 man-hours	

9. Budget

The proposed project, **"Sudoku Master Pro: An Interactive Sudoku Game and Learning Platform,"** will be developed using the personal resources of the project team. The development environment consists of personal laptops and freely available software tools, including the C programming language, GCC Compiler, Code::Blocks IDE, and GitHub for version control. Since all required software and tools are open-source and readily accessible, no additional expenditure is necessary for software licenses, hardware purchases, or third-party services.

Project Item	Amount	Unit Price	Total
Software license	0	BDT 0	BDT 0
Personal Laptop	3	Existing	BDT 0
Server License	0	N/A	BDT 0
Database license	0	N/A (file-based)	BDT 0
Development Consultant	0	Faculty (no cost)	BDT 0
Project Management	0	N/A	BDT 0
Training (Time + Materials)	0	N/A	BDT 0

10. Development Team & Stakeholders

Team Structure

Id	Name	Designation	Responsibilities
20254103248	OALID KHAN	Team Leader & Core Programmer	Overall coordination, authentication module, game loop, tutorial section, final integration and testing
20254103245	MUNTAHA MOU JIM	Back-end Programmer	Backtracking solver, hint system, scoring engine, save/load file I/O, leaderboard
20254103257	ANTAR CHANDRA KAR	Back-end Programmer	Board ASCII display, statistics module, cross-platform testing, documentation

Stakeholders

Stakeholder	Type	Interest / Role
Student Developers (3)	Internal	Build, test, and submit the application
Faculty Supervisor	Internal	Guide development, evaluate and grade the submission
University (BUBT, CSE Dept.)	External	Academic context; requires SDP completion for credit
Future Students / Learners	External	Use as reference for C programming projects
General Sudoku Players	External	End users who play the game after release

11. Limitations & Future Works

Current Limitations:

- Console-only interface: no graphical display; visual appeal is limited to ASCII art.
- Fixed puzzle bank: contains 15 hand-curated puzzles (5 per difficulty); a truly random generator may produce puzzles with multiple solutions.
- No network support: high-score competition is limited to a single machine's leaderboard file.
- Password storage is plain-text in the binary file; no hashing or encryption is applied.
- Maximum of 50 registered users and 5 save slots; limited by fixed-size struct arrays.
- No undo/redo functionality during a game session.

Future Works:

- Implement a procedural Sudoku generator that creates unique, solvable puzzles at runtime for each game, ensuring infinite replayability.
- Add password hashing (e.g., SHA-256 via OpenSSL) for secure credential storage.
- Introduce an undo/redo stack to allow players to revert incorrect moves without incurring a penalty.
- Port the UI to an ncurses or SDL2 front-end for colour display, mouse support, and a modern look while keeping the C back-end unchanged.
- Implement a network leaderboard using POSIX sockets so players on different machines can compete on a shared scoreboard.

- Add an auto-note (candidate) system that automatically marks possible numbers in empty cells to assist players learning Sudoku logic.

12. References

[1] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ: Prentice Hall, 1988.

[2] D. E. Knuth, "Dancing Links," in Millennial Perspectives in Computer Science, Palgrave, 2000, pp. 187-214. Available: https://arxiv.org/abs/cs/0011047

[3] T. Mantere and J. Koljonen, "Solving and Rating Sudoku Puzzles with Artificial Intelligence," in Proc. New Trends in Intelligent Information Processing and Web Mining, Springer, 2006, pp. 323-332.

[4] GCC, The GNU Compiler Collection, Free Software Foundation. [Online]. Available: https://gcc.gnu.org/

[5] Sudoku.com, Easybrain. [Online]. Available: https://sudoku.com/ [Accessed: Jun. 2025].

[6] GNOME Project, "GNOME Sudoku." [Online]. Available: https://wiki.gnome.org/Apps/Sudoku [Accessed: Jun. 2025].

[7] ncurses-sudoku, GitHub. [Online]. Available: https://github.com/AlexeyD/ncurses-sudoku [Accessed: Jun. 2025].

[8] Cloud Security Alliance (2009), Quantum-safe Security Working Group. [Online]. Available: https://cloudsecurityalliance.org/group/quantum-safe-security/